CS341 · COMPUTER GRAPHICS

Output Primitives up to DDA

ملخص الميد · Chapter 3 & 5

محارب

محتوى الملف

01 Output Primitives و OpenGL

02 glVertex و glBegin

03 برنامج OpenGL كامل (مثال السلايد)

04 أنظمة الإحداثيات

05 معادلة الخط

06 خوارزمية DDA

01 OpenGL و Output Primitives

Graphics output primitives: functions in a graphics package used to describe the various picture components. Primitives that describe the geometry of objects are called **geometric primitives**.
Basic primitives: points, straight lines, circles, conic sections, curves, surfaces, polygons and filled color areas.

OpenGL: مكتبة رسم 3D مستقلة عن الجهاز (cross-platform) تستفيد من كرت الشاشة. هي CG API.
تنقسم ثلاث مكتبات:

المكتبة	الملفات	Header	Prefix
OpenGL (core)	opengl32.lib / .dll	gl.h	gl
Utility (glu)	glu32.lib / .dll	glu.h	glu
Toolkit (glut)	glut32.lib / .dll	glut.h	glut

- أسماء الدوال تبدأ بـ **gl**: glVertex، glClear، glBegin.
- الثوابت: GL_POLYGON، GL_RGB، GL_LINES، GL_POINTS.
- أنواع البيانات: GLboolean، GLdouble، GLfloat، GLint.
- glut مو جزء من ويندوز، ينزل مجاناً.

02 glVertex و glBegin

```
glVertex2i(4, 5);            2 integer coordinates (z = 0)
```

```
glVertex3f(2.0, 4.1, 6.0);   3 float coordinates
```

```
glVertex3fv(vector);         v = pass an array
```

الرأس (vertex) ينكتب بين **glBegin(mode)** و **glEnd()**، وللمضلعات عادة بعكس عقارب الساعة. اللون الحالي (**glColor3f**) ينطبق على كل الرؤوس اللي بعده. اللون الافتراضي أبيض، وحجم النقطة الافتراضي بكسل واحد.

النوع	Modes
Points	GL_POINTS
Lines	GL_LINES, GL_LINE_STRIP, GL_LINE_LOOP
Triangles	GL_TRIANGLES, GL_TRIANGLE_STRIP, GL_TRIANGLE_FAN
Quads	GL_QUADS, GL_QUAD_STRIP
Polygons	GL_POLYGON

مع خمس نقاط p1 ... p5:

Mode	النتيجة
GL_LINES	خط p1–p2 وخط p3–p4، و p5 تنهمل
GL_LINE_STRIP	خطوط متصلة (polyline) p1–p2–p3–p4–p5
GL_LINE_LOOP	نفس الـ strip ويقفل p5–p1

03 برنامج OpenGL كامل (مثال السلايد)

```
#include <GL/glut.h>
void init (void) {
  glClearColor (1.0, 1.0, 1.0, 0.0);      // window color white (4th = alpha)
  glMatrixMode (GL_PROJECTION);           // set projection parameters
  gluOrtho2D (0.0, 200.0, 0.0, 150.0);    // 2D orthogonal projection
}
void lineSegment (void) {
  glClear (GL_COLOR_BUFFER_BIT);          // clear the display window
  glColor3f (1.0, 0.0, 0.0);              // red
  glBegin (GL_LINES);
    glVertex2i (180, 15);
    glVertex2i (10, 145);
  glEnd ( );
  glFlush ( );                            // process all routines quickly
}
void main (int argc, char** argv) {
  glutInit (&argc, argv);
  glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
  glutInitWindowPosition (50, 100);       // top-left corner of the window
  glutInitWindowSize (400, 300);          // width, height
  glutCreateWindow ("An Example OpenGL Program");
  init ( );
  glutDisplayFunc (lineSegment);
  glutMainLoop ( );                       // display everything and wait
}
```

04 أنظمة الإحداثيات

النظام	معناه
World coordinates	نظام كارتيزي نوصف فيه الأجسام (2D أو 3D). خط = نقطتين، مضلع = رؤوسه.
Screen coordinates	أعداد صحيحة = مواقع البكسلات في الـ frame buffer. y = رقم الـ scan line و x = رقم العمود. الهاردوير يبدأ من الزاوية العلوية اليسرى.

دوال منخفضة المستوى: **setPixel(x, y)** تخزن اللون الحالي في البكسل، و **getPixel(x, y, color)** تقرأ لون البكسل. في 3D نخزن عمق (depth) كذلك، وفي 2D العمق صفر.

Absolute vs Relative

Absolute	Relative
القيم هي المواقع الفعلية في نظام الإحداثيات (بالنسبة للأصل).	إزاحة (offset) من آخر موقع استخدمناه (current position).

```
current position = (3, 8),  relative (2, -1)  →  absolute (5, 7)
```

يتكرر في الميد

What is the difference between absolute and relative coordinates? (472, 461)

gluOrtho2D

```
glMatrixMode (GL_PROJECTION);
```

```
glLoadIdentity ();
```

```
gluOrtho2D (xmin, xmax, ymin, ymax);
```

تحدد نظام إحداثيات 2D: `(xmin, ymin)` تحت يسار و `(xmax, ymax)` فوق يمين. اللي داخل الحدود يظهر، واللي برا ينقص. كل المواقع لازم تكون absolute.

Coordinate extents: أقل وأكبر x و y و z للجسم، وتسمى **bounding box** (وفي 2D **bounding rectangle**).

05 معادلة الخط

الشاشة ما تقدر ترسم خط مثالي؛ نقرّب كل نقطة لأقرب بكسل، فيطلع الخط مدرّج (stair-step / jaggies) إلا الأفقي والرأسي.

$$y = m \cdot x + b$$

$$m = \frac{yend - y0}{xend - x0}$$

$$b = y0 - m \cdot x0$$

$$\delta y = m \cdot \delta x \qquad \delta x = \frac{\delta y}{m}$$

الميل	نخطو بـ
`|m| < 1`	خطوة وحدة، ونحسب y x
`|m| > 1`	خطوة وحدة، ونحسب x y
`m = 1`	`δx = δy`

06 خوارزمية DDA

DDA (Digital Differential Analyzer): floating-point increments, then rounded to integers. Rounding of floating-point values makes pixels **drift away** from the true line for long segments, and rounding is **time consuming**.

Case 1: $|m| \leq 1$ x is the variable: $\Delta x = 1 \rightarrow y(k+1) = y(k) + m$

Case 2: $|m| > 1$ y is the variable: $\Delta y = 1 \rightarrow x(k+1) = x(k) + \frac{1}{m}$

الخطوات

```
dx = xEnd − x0          dy = yEnd − y0
```

```
steps = max(|dx|, |dy|)
```

$$xInc = \frac{dx}{steps} \qquad yInc = \frac{dy}{steps}$$

```
plot round(x0), round(y0)
```

```
repeat steps times:  x += xInc,  y += yInc,  plot (round(x), round(y))
```

```
round(a) = int(a + 0.5)
```

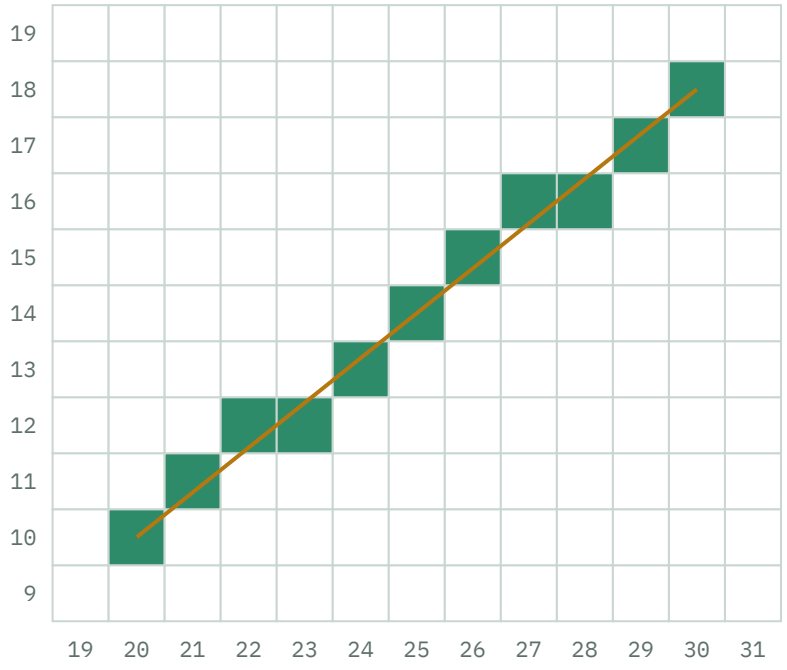
```
void lineDDA (int x0, int y0, int xEnd, int yEnd) {
  int dx = xEnd - x0, dy = yEnd - y0, steps, k;
  float xIncrement, yIncrement, x = x0, y = y0;
  if (fabs (dx) > fabs (dy)) steps = fabs (dx);
  else steps = fabs (dy);
  xIncrement = float (dx) / float (steps);
  yIncrement = float (dy) / float (steps);
  setPixel (round (x), round (y));
  for (k = 0; k < steps; k++) {
    x += xIncrement;  y += yIncrement;
    setPixel (round (x), round (y));
  }
}
```

مثال السلايد: (20,10) إلى (30,18)

$$dx = 10,\quad dy = 8,\quad m = \frac{8}{10} = 0.8 \;\rightarrow\; |m| < 1$$

$$steps = 10,\quad xInc = 1,\quad yInc = 0.8$$

k	x	y	round(x)	round(y)
0	20	10	20	10
1	21	10.8	21	11
2	22	11.6	22	12
3	23	12.4	23	12
4	24	13.2	24	13
5	25	14	25	14
6	26	14.8	26	15
7	27	15.6	27	16
8	28	16.4	28	16
9	29	17.2	29	17
10	30	18	30	18



مثال 472: (2,2) إلى (4,10) — ميل أكبر من 1

$$dx = 2, \quad dy = 8, \quad m = \frac{8}{2} = 4 \quad \rightarrow \quad |m| > 1$$

$$steps = 8, \quad xInc = \frac{2}{8} = 0.25, \quad yInc = 1$$

k	x	y	round(x)	round(y)
0	2	2	2	2
1	2.25	3	2	3
2	2.5	4	3	4
3	2.75	5	3	5
4	3	6	3	6
5	3.25	7	3	7
6	3.5	8	4	8
7	3.75	9	4	9
8	4	10	4	10